Git & GitHub — Production-Grade Notes for Backend Developers

> ***Audience:*** *Backend developers with 2+ years of experience working in team environments with large-scale, centralized/monorepo setups.* ***Goal:*** *Practical, day-to-day reference — not a tutorial. Treat this as internal team documentation.*

Table of Contents

1. Core Git Concepts
2. Daily Workflow
3. Branching Strategies
4. Pull Requests (PRs) in Teams
5. Merging & Rebasing
6. Conflict Resolution
7. Advanced Git Commands
8. Debugging & Recovery
9. Working with Large Teams (Single Repo)
10. GitHub Practical Usage
11. Real-World Scenarios
12. Best Practices & Common Mistakes
13. Command Cheat Sheet

---

1. Core Git Concepts

The Three Zones — Always Keep This in Mind

```
Working Directory  →  Staging Area (Index)  →  Local Repository  →  Remote
      (edit)               (git add)               (git commit)      (git push)
```

Zone	What it is	Real meaning
Working Directory	Files on your disk	What you're currently editing
Staging Area	A snapshot prep area	What will go into the next commit
Local Repo	`.git` folder on your machine	Full history, local only
Remote	GitHub / GitLab server	Shared source of truth for the team

HEAD — What It Actually Is

`HEAD` is a pointer. It points to the commit you're currently "on."

```
cat .git/HEAD
# → ref: refs/heads/feature/payment-service   (on a branch)
# → a3f9c1d...                                 (detached HEAD — dangerous zone)
```

- **Detached HEAD** happens when you `git checkout <commit-hash>`. You're not on any branch. Any commit you make here will be lost unless you create a branch from it immediately.

How Git Tracks Changes (High-Level, Useful Understanding)

Git doesn't store diffs — it stores **snapshots**. Each commit is a snapshot of all tracked files at that point in time, plus a pointer to its parent commit.

```
A ← B ← C ← D    (main)
              ↑
             HEAD
```

- Git uses **SHA-1 hashes** (40-char strings like `a3f9c1d...`) to identify every commit, tree, and blob.
- Two identical files → same hash → Git stores only one copy. This is how large repos stay manageable.
- **Branches are just pointers** to a commit. Creating a branch is nearly free (just writes a 41-byte file).

```
# See what a branch pointer actually is
cat .git/refs/heads/main
# → a3f9c1d8e2b7f4a1c9d0e3f6b8a2c5d7e9f1b3a4
```

---

2. Daily Workflow

The Standard Developer Loop

```
# 1. Start your day — sync with remote main
git checkout main
git pull origin main

# 2. Create a feature branch
git checkout -b feature/JIRA-123-add-user-auth

# 3. Do your work, then stage selectively
git add src/auth/login.go          # stage specific file
git add src/auth/                  # stage entire directory
git add -p                         # stage hunks interactively (recommended)

# 4. Commit with a meaningful message
git commit -m "feat(auth): add JWT token validation middleware"

# 5. Push to remote
git push origin feature/JIRA-123-add-user-auth

# 6. Keep branch updated with main (do this daily)
git fetch origin
git rebase origin/main
```

Writing Meaningful Commit Messages

Follow **Conventional Commits** — it's the industry standard and works with automated changelogs and CI tools.

Format:

```
<type>(<scope>): <short description>

[optional body — what and WHY, not HOW]

[optional footer: BREAKING CHANGE, closes #issue]
```

Types:

Type	When to use
`feat`	New feature
`fix`	Bug fix
`refactor`	Code restructure, no behavior change
`perf`	Performance improvement
`chore`	Build tooling, dependency updates
`docs`	Documentation only
`test`	Adding or fixing tests
`ci`	CI/CD config changes

Type	When to use
`revert`	Reverting a previous commit

Real examples:

```
# Good
git commit -m "fix(payments): handle Stripe webhook retry with idempotency key"
git commit -m "feat(api): add rate limiting middleware for /auth endpoints"
git commit -m "refactor(db): extract connection pooling into separate package"
git commit -m "chore(deps): upgrade go-redis from v8 to v9"

# Bad — these are useless in git log
git commit -m "fix bug"
git commit -m "changes"
git commit -m "wip"
git commit -m "asdfgh"
```

Multi-line commit (for complex changes):

```
git commit -m "feat(orders): implement distributed order locking

Used Redis SETNX for distributed lock to prevent duplicate order
processing under concurrent requests. Lock TTL set to 30s.

Closes #JIRA-456
BREAKING CHANGE: OrderService.Create() now requires Redis client injection"
```

Handling Frequent Updates from Main

```
# Option A: Rebase your branch onto main (keeps history clean — preferred)
git fetch origin
git rebase origin/main

# If conflicts arise during rebase
# Fix the conflict → then:
git add <resolved-file>
git rebase --continue

# Abort if things go wrong
git rebase --abort

# Option B: Merge main into your branch (creates a merge commit — noisier)
git merge origin/main
```

> ***Rule of thumb:*** *Use rebase for personal feature branches. Use merge for integrating into shared/long-lived branches.*

---

3. Branching Strategies

Feature Branch Workflow (Default for Most Teams)

Every piece of work — features, fixes, hotfixes — gets its own branch. `main` is always deployable.

```
main ──────────────────────────────────────────────────▶
       └── feature/A ─── PR ──┘
                 └── feature/B ─── PR ──┘
                       └── fix/C ─── PR ──┘
```

Git Flow vs Trunk-Based Development

	Git Flow	Trunk-Based
Branches	`main`, `develop`, `feature/*`, `release/*`, `hotfix/*`	`main` + short-lived feature branches
Release cycle	Scheduled releases	Continuous deployment
Team size	Good for larger teams with release windows	Good for mature CI/CD teams
Complexity	Higher — more branch management	Lower — simpler workflow
Use when	Apps with versioned releases (mobile apps, SDKs)	Microservices, SaaS with daily deploys

Git Flow in practice (when you need it):

```
# Start a feature
git checkout develop
git checkout -b feature/JIRA-789-notifications

# Start a hotfix against production
git checkout main
git checkout -b hotfix/critical-sql-injection-fix

# Merge hotfix to both main AND develop
git checkout main && git merge hotfix/critical-sql-injection-fix
git checkout develop && git merge hotfix/critical-sql-injection-fix
```

Trunk-based in practice:

```
# Branch off main, keep it short-lived (< 2 days ideally)
git checkout main
git checkout -b feat/JIRA-101-add-healthcheck

# Push frequently, merge via PR quickly
# Use feature flags in code if the feature isn't ready to expose
```

Branch Naming Conventions

```
feature/JIRA-123-short-description
fix/JIRA-456-payment-null-pointer
hotfix/prod-broken-auth-header
chore/upgrade-postgres-driver
refactor/user-service-clean-up
release/v2.4.0
```

Rules:

- Always lowercase, use hyphens (not underscores)
- Include the ticket number if your team uses a tracker (Jira, Linear, etc.)
- Keep description short but meaningful
- Never use spaces or special characters

---

4. Pull Requests (PRs) in Teams

How to Create a Good PR

A PR is a communication tool, not just a code submission.

PR Checklist before opening:

- ☐ Branch is rebased on latest `main`
- ☐ All tests pass locally
- ☐ No debug logs, commented-out code, or `TODO`s without context
- ☐ Self-reviewed your own diff first
- ☐ Description explains *what* and *why*

PR Description Template:

```
## What
Brief description of what this PR does.

## Why
Why is this change needed? Link to ticket.

## How
High-level approach taken.

## Testing
How was this tested? Any edge cases covered?

## Screenshots / Logs (if applicable)

## Checklist
- [ ] Tests added/updated
- [ ] Docs updated (if needed)
- [ ] No breaking changes (or documented if yes)
```

Keep PRs small. A PR with 50 lines of changed code gets 10x better review than one with 500 lines. Split large features into stacked PRs if needed.

Code Review Best Practices

As a reviewer:

```
# Constructive (Good)
"Consider using a context with timeout here to prevent goroutine leaks
if the DB call hangs. Something like: ctx, cancel := context.WithTimeout(...)"

# Blocking nitpick (Bad)
"This variable name is bad."

# Clear severity markers (use these)
[nit] Minor style thing — feel free to ignore
[suggestion] Consider this, but your call
[question] I don't understand this — can you explain?
[blocker] This will cause a bug / security issue — must fix before merge
```

As the PR author:

- Don't take comments personally — code review is about the code.
- Respond to every comment (either fix it or explain why you didn't).
- Use "Resolved" only after you've actually addressed it.

Squash vs Merge vs Rebase Merge

Strategy	History	Use when
Merge commit	Preserves all commits + adds a merge commit	Long-lived branches, want full history
Squash and merge	All commits squashed into one on main	Feature branches with messy WIP commits
Rebase and merge	Replays commits on top of main, no merge commit	Clean linear history needed

In most teams: Use **Squash and merge** for feature branches. It keeps `main` history clean and each feature is one atomic commit.

```
# What main looks like after squash merge:
main: A → B → C → [feat: add payment service] → [fix: handle webhook retry]

# vs regular merge (noisy):
main: A → B → C → wip → wip2 → fix → fix2 → Merge branch 'feature/payments'
```

---

5. Merging & Rebasing

Merge vs Rebase — The Real Difference

Merge creates a new commit that ties together two branch histories:

```
main:    A ── B ── C ──────────── M  (merge commit)
                   \            /
feature:            D ── E ── F
```

Rebase replays your commits on top of another branch — rewrites history:

```
main:    A ── B ── C
                   \
feature:            D' ── E' ── F'   (new commits, same changes)
```

When to Use Each

Scenario	Use
Updating personal feature branch with main	`rebase`
Merging a feature PR into main	Depends on team convention (squash merge is common)
Long-lived shared branch (like `develop`)	`merge` — don't rebase shared branches
Cleaning up commits before PR	`rebase -i` (interactive)
Hotfix into production	`merge` (want an explicit merge commit for audit trail)

Interactive Rebase — Cleaning Up Commit History

Before raising a PR, clean your commits:

```
# Rebase and edit last 4 commits
git rebase -i HEAD~4
```

This opens an editor:

```
pick a1b2c3d feat(auth): initial JWT middleware
pick d4e5f6g wip
pick h7i8j9k fix typo
pick k1l2m3n actually fix the bug
```

Change to:

```
pick a1b2c3d feat(auth): initial JWT middleware
squash d4e5f6g wip
squash h7i8j9k fix typo
squash k1l2m3n actually fix the bug
```

Result: one clean commit with a proper message.

Commands available in interactive rebase:

Command	What it does
`pick`	Keep commit as-is
`squash` / `s`	Merge into previous commit
`fixup` / `f`	Like squash, discard this commit's message
`reword` / `r`	Keep commit, edit message
`drop` / `d`	Delete commit entirely
`edit` / `e`	Pause and amend the commit

The Golden Rule of Rebasing

> ***Never rebase a branch that others are working on.***

If you rebase `feature/payments` and someone else already has it checked out — their branch will diverge. Only rebase your own local/personal branches.

---

6. Conflict Resolution

Step-by-Step Conflict Resolution

```
# 1. You're rebasing and hit a conflict
git rebase origin/main
# CONFLICT (content): Merge conflict in src/user/service.go

# 2. Open the file — Git marks conflicts like this:
<<<<<<< HEAD (your current branch / main)
func GetUser(id string) (*User, error) {
    return db.FindByID(id)
}
=======
func GetUser(ctx context.Context, id string) (*User, error) {
    return db.FindByIDWithContext(ctx, id)
}
>>>>>>> feature/context-propagation

# 3. Manually resolve — keep what's correct:
func GetUser(ctx context.Context, id string) (*User, error) {
    return db.FindByIDWithContext(ctx, id)
}

# 4. Stage the resolved file
git add src/user/service.go

# 5. Continue the rebase
git rebase --continue

# 6. If there are more conflicts, repeat steps 2-5
```

Using a Merge Tool

```
# Configure VS Code as merge tool
git config --global merge.tool vscode
git config --global mergetool.vscode.cmd 'code --wait $MERGED'

# Launch it
git mergetool
```

Real Example: Two Devs Modified the Same Config File

Developer A changed `config/database.yaml`:

```
max_connections: 50
timeout: 30s
```

Developer B changed the same file:

```
max_connections: 100
pool_size: 10
```

Conflict:

```
<<<<<<< HEAD
max_connections: 50
timeout: 30s
=======
max_connections: 100
pool_size: 10
>>>>>>> feature/db-pool-tuning
```

Resolution (combine both intentional changes):

```
max_connections: 100
timeout: 30s
pool_size: 10
```

Best Practices to Minimize Conflicts

1. **Pull/rebase daily** — the longer you wait, the worse conflicts get.
2. **Break work into small PRs** — smaller diffs = fewer conflicts.
3. **Communicate with your team** — "I'm refactoring `user_service.go` today, heads up."
4. **Don't refactor + feature in the same PR** — mixes concerns and causes broader diffs.
5. **Use code ownership** — if only one team owns a file, conflicts are rare.
6. **Config files are conflict hotspots** — coordinate changes or use environment-specific files.

---

7. Advanced Git Commands

`git stash` — Your Temporary Shelf

Use when you need to switch context without committing half-done work.

```
# Save current changes
git stash

# Save with a meaningful name
git stash push -m "half-done payment retry logic"

# List stashes
git stash list
# stash@{0}: On feature/payments: half-done payment retry logic
# stash@{1}: WIP on fix/auth: debug logging

# Apply latest stash (keeps it in stash list)
git stash apply

# Apply specific stash
git stash apply stash@{1}

# Apply and remove from stash
git stash pop

# Drop a specific stash
git stash drop stash@{0}

# Stash including untracked files
git stash push -u
```

Real scenario: You're mid-feature when someone asks you to review and test their PR. Stash your work, switch to their branch, test it, come back and pop.

`git cherry-pick` — Grab a Specific Commit

```
# Apply a single commit from another branch to current branch
git cherry-pick a3f9c1d

# Apply a range of commits
git cherry-pick a3f9c1d..b4e8d2f

# Cherry-pick without committing (stage changes only)
git cherry-pick --no-commit a3f9c1d
```

Real scenario: A critical fix was merged into `develop` but you need it in `main` immediately without merging all of `develop`. Cherry-pick that specific fix commit.

`git reset` vs `git revert`

This is one of the most important distinctions in team settings.

	`git reset`	`git revert`
What it does	Moves HEAD backward (rewrites history)	Creates a new commit that undoes changes
Safe for remote?	**NO** — never on pushed commits	**YES** — safe for shared branches
Use when	Local commits you haven't pushed	Commits already on remote/shared branch

```
# reset — local only, dangerous on remote
git reset --soft HEAD~1    # Undo last commit, keep changes staged
git reset --mixed HEAD~1   # Undo last commit, keep changes unstaged (default)
git reset --hard HEAD~1    # Undo last commit, DISCARD all changes permanently

# revert — safe for shared branches
git revert HEAD            # Creates new commit undoing last commit
git revert a3f9c1d         # Reverts a specific commit
git revert HEAD~3..HEAD    # Reverts last 3 commits
```

`git reflog` — The Ultimate Safety Net

`reflog` records every movement of HEAD — even after resets, rebases, or deletions. It's local only (30-day default expiry).

```
git reflog

# Output:
# a3f9c1d HEAD@{0}: rebase finished: returning to refs/heads/feature/auth
# b4e8d2f HEAD@{1}: rebase: feat(auth): add JWT validation
# d5f1a3b HEAD@{2}: checkout: moving from main to feature/auth
# e6g2b4c HEAD@{3}: reset: moving to HEAD~1   ← "I accidentally reset here!"
# f7h3c5d HEAD@{4}: commit: feat(auth): complete auth module

# Recover lost commit
git checkout f7h3c5d          # Go to lost commit
git checkout -b recovery/auth-module  # Create branch from it
```

`git clean` — Remove Untracked Files

```
# See what would be removed (dry run — always do this first)
git clean -n

# Remove untracked files
git clean -f

# Remove untracked files AND directories
git clean -fd

# Remove ignored files too (careful — removes .env, build artifacts, etc.)
git clean -fdx
```

---

8. Debugging & Recovery

Recover a Deleted Branch

```
# Find the last commit of the deleted branch
git reflog

# a3f9c1d HEAD@{5}: checkout: moving from feature/payments to main

# Recreate the branch
git checkout -b feature/payments a3f9c1d
```

Fix a Wrong Commit Pushed to Remote

Scenario: You committed a secret key or pushed broken code to a shared branch.

```
# Option 1: Revert (safe — adds a new "undo" commit)
git revert HEAD
git push origin main

# Option 2: If you MUST rewrite history (coordinate with team first)
git reset --hard HEAD~1
git push --force-with-lease origin feature/my-branch
# Use --force-with-lease not --force — it fails if someone else pushed since your last
fetch
```

> ***Never force-push to*** `main` ***or*** `develop` *unless protected branch rules are temporarily disabled and the whole team is aware.*

Find Which Commit Introduced a Bug — `git bisect`

```
# Start bisect
git bisect start

# Mark current state as bad
git bisect bad

# Mark a known good commit (e.g., last week's release)
git bisect good v2.3.0

# Git will checkout a commit in the middle
# Test your code → then tell git if it's good or bad
git bisect good    # or
git bisect bad

# Git narrows it down — eventually identifies the culprit commit
# When done:
git bisect reset
```

Find Who Changed a Line — `git blame`

```
# See who last modified each line of a file
git blame src/payments/processor.go

# Show specific line range
git blame -L 45,60 src/payments/processor.go

# Ignore whitespace changes
git blame -w src/payments/processor.go
```

Find Which Commit Introduced a String — `git log -S`

```
# Find commits that added or removed a specific string
git log -S "stripe_secret_key" --all

# Search commit messages
git log --grep="payment" --oneline
```

Undo a Merge Commit

```
# If not pushed yet
git reset --hard HEAD~1

# If already pushed — revert the merge commit
git revert -m 1 <merge-commit-hash>
# -m 1 tells Git to use the first parent (main branch) as mainline
```

---

9. Working with Large Teams (Single Repo)

Branch Protection Rules (GitHub Settings)

Set these on `main` (and `develop` if using Git Flow):

- ✅ **Require pull request reviews before merging** (minimum 1-2 approvals)
- ✅ **Require status checks to pass** (CI must be green)
- ✅ **Require branches to be up to date before merging**
- ✅ **Restrict who can push to matching branches**
- ✅ **Do not allow force pushes**
- ✅ **Do not allow deletions**

CODEOWNERS File

Automatically request reviews from the right team when specific files/directories are changed.

```
# .github/CODEOWNERS

# Global owner
*                           @org/backend-team

# Payments service — always reviewed by payments team
/services/payments/         @org/payments-team

# Infrastructure configs
/infrastructure/            @org/devops-team
/docker-compose.yml         @org/devops-team

# CI/CD pipelines
/.github/workflows/         @org/devops-team

# Shared proto files — two approvals needed
/proto/                     @org/platform-team @org/api-team
```

Avoiding Breaking `main`

1. **All work goes through PRs** — direct pushes to `main` blocked by branch protection.
2. **CI must pass** before merge — unit tests, lint, build.
3. **One approval minimum** before merge.
4. **Use feature flags** for incomplete features that need to be deployed but not activated.
5. **Semantic versioning** on releases — tag every release (`git tag v2.4.0`).

CI/CD Integration with GitHub Actions

```yaml
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Go
        uses: actions/setup-go@v5
        with:
          go-version: '1.22'

      - name: Run tests
        run: go test ./... -race -coverprofile=coverage.out

      - name: Run lint
        uses: golangci/golangci-lint-action@v4

      - name: Build
        run: go build ./...
```

10. GitHub Practical Usage

Repository Structure Best Practices

```
my-service/
├── .github/
|   ├── workflows/          # CI/CD pipelines
|   ├── CODEOWNERS          # Auto-assign reviewers
|   ├── PULL_REQUEST_TEMPLATE.md
|   └── ISSUE_TEMPLATE/
|       ├── bug_report.md
|       └── feature_request.md
├── src/
├── tests/
├── docs/
├── scripts/                # Build, migration, setup scripts
├── .gitignore
├── README.md
└── CHANGELOG.md
```

`.gitignore` for Backend Projects

```
# Env files
.env
.env.*
!.env.example         # Keep the example file

# Build artifacts
bin/
dist/
*.exe

# IDE
.vscode/
.idea/

# OS
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Dependencies
vendor/               # Go vendor (if not committed)
node_modules/

# Test artifacts
coverage.out
*.test

# Secrets (extra safety net - should not be here in the first place)
*.pem
*.key
*_rsa
```

Pull Request Template

```
<!-- .github/PULL_REQUEST_TEMPLATE.md -->

## Summary
<!-- What does this PR do? -->

## Related Issue
<!-- Closes #JIRA-XXX or link to ticket -->

## Type of Change
- [ ] Bug fix
- [ ] New feature
- [ ] Refactor
- [ ] Performance improvement
- [ ] Docs / chore

## How to Test
<!-- Steps to verify this works -->

## Checklist
- [ ] Tests added/updated
- [ ] No secrets or credentials in code
- [ ] Self-reviewed the diff
- [ ] Breaking changes documented
```

Issues, Labels, and Milestones

Useful label system:

Label	Color	Purpose
`bug`	🔴 Red	Something is broken
`feature`	🟢 Green	New feature request
`performance`	🟠 Orange	Performance issue
`security`	⚫ Black	Security vulnerability
`blocked`	🔴 Red	Waiting on dependency
`in-review`	🔵 Blue	PR opened, under review
`wont-fix`	⚪ Gray	Acknowledged, won't address
`good-first-issue`	💜 Purple	Good for new team members

Milestones = group issues into a release: `v2.4.0 – Payments Overhaul` → attach all related issues and PRs.

GitHub Actions — Real Use Cases

```yaml
# Auto-assign PR reviewer based on changed files
name: Auto Assign
on: [pull_request]
jobs:
  assign:
    runs-on: ubuntu-latest
    steps:
      - uses: kentaro-m/auto-assign-action@v1.2.5

---

# Deploy to staging on merge to main
name: Deploy Staging
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy
        run: ./scripts/deploy.sh staging
        env:
          SSH_KEY: ${{ secrets.DEPLOY_SSH_KEY }}

---

# Auto-comment on PR with coverage report
name: Coverage
on: [pull_request]
jobs:
  coverage:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: go test ./... -coverprofile=coverage.out
      - uses: codecov/codecov-action@v4
```

---

11. Real-World Scenarios

Scenario 1: You Pushed Wrong Code to `main`

```
# Check what was pushed
git log origin/main --oneline -5

# Safe option — revert the bad commit
git revert <bad-commit-hash>
git push origin main

# If it was a merge commit
git revert -m 1 <merge-commit-hash>
git push origin main

# Nuclear option — only if no one has pulled it yet (coordinate with team)
git reset --hard HEAD~1
git push --force-with-lease origin main
```

Scenario 2: Two Developers Modified the Same File

Developer A and Developer B both edited `src/user/handler.go`. Dev A merged first.

```
# Dev B updates their branch
git fetch origin
git rebase origin/main

# Conflict appears in handler.go
# Open file, resolve conflict manually
# Keep both changes if they don't logically conflict
# Consult Dev A if unsure which change should win

git add src/user/handler.go
git rebase --continue
git push --force-with-lease origin feature/dev-b-branch
```

Scenario 3: Need to Move a Commit to Another Branch

You accidentally committed to `main` instead of your feature branch.

```
# Step 1: Note the commit hash
git log --oneline -3
# a3f9c1d feat: add rate limiter   ← want this on feature branch

# Step 2: Create / switch to the correct branch
git checkout feature/rate-limiter
git cherry-pick a3f9c1d

# Step 3: Remove the commit from main
git checkout main
git reset --hard HEAD~1          # Removes last commit locally
git push --force-with-lease origin main  # Only if no one pulled yet
```

Scenario 4: Accidentally Deleted a Branch

```
# Find the last commit of the deleted branch via reflog
git reflog --all | grep "feature/payments"
# a3f9c1d refs/heads/feature/payments@{0}: commit: feat: add stripe integration

# Recreate it
git checkout -b feature/payments a3f9c1d
git push origin feature/payments
```

Scenario 5: Need to Undo the Last 3 Commits (Not Pushed)

```
# Keep the changes but uncommit them
git reset --soft HEAD~3

# All 3 commits' changes are now staged
# You can now re-commit them differently or just unstage
```

Scenario 6: Committed Secrets/Credentials Accidentally

```
# 1. Remove from latest commit (if not pushed)
git reset --soft HEAD~1
# Remove the secret from the file
git add .
git commit -m "chore: remove accidentally added config"

# 2. If already pushed — IMMEDIATELY rotate the credentials
# Then use BFG Repo Cleaner or git filter-repo to purge from history
git filter-repo --path config/secrets.yaml --invert-paths

# 3. Force push all branches (coordinate with team)
git push --force --all

# ⚠️ Purging from GitHub: the file may still be in PR history/forks
# Contact GitHub support if the secret is highly sensitive
```

Scenario 7: Your Feature Branch Is Way Behind Main (100+ commits)

```
# Don't panic — rebase handles this
git fetch origin
git rebase origin/main

# Expect multiple conflict rounds if files overlap heavily
# Use --strategy-option=theirs for auto-resolving if you know your changes win
git rebase -X theirs origin/main

# Or resolve manually commit-by-commit
# After each conflict: git add . && git rebase --continue
```

Scenario 8: You Need the Same Fix on Multiple Release Branches

```
# Your fix is on main as commit a3f9c1d
# Need it on release/v2.3 and release/v2.4

git checkout release/v2.3
git cherry-pick a3f9c1d
git push origin release/v2.3

git checkout release/v2.4
git cherry-pick a3f9c1d
git push origin release/v2.4
```

Scenario 9: Investigating a Production Bug — Find When It Was Introduced

```
# Use git bisect
git bisect start
git bisect bad HEAD              # current version is broken
git bisect good v2.3.0           # this version was fine

# Git checks out midpoint commit
# Run your test / reproduce the bug
# If broken:
git bisect bad
# If fine:
git bisect good

# Git will eventually output:
# "a3f9c1d is the first bad commit"

git bisect reset                 # Back to HEAD
git show a3f9c1d                 # Inspect the culprit
```

Scenario 10: Teammate Force-Pushed and Your Local Branch Is Outdated

```
# Your local branch diverged from remote after their force-push
git fetch origin
git status
# "Your branch and 'origin/feature/payments' have diverged"

# If you haven't made changes on top of the old commits
git reset --hard origin/feature/payments

# If you have new commits on top of old ones
git rebase origin/feature/payments
```

Scenario 11: You Need to See What Changed in a Specific PR

```
# Fetch the PR branch
git fetch origin pull/42/head:pr-42
git checkout pr-42

# Or compare directly
git diff main...feature/payment-service

# See all files changed
git diff --name-only main...feature/payment-service
```

Scenario 12: Merge Went Wrong — Repo Is in a Bad State

```
# During a bad merge, abort it cleanly
git merge --abort

# During a bad rebase, abort it
git rebase --abort

# If you committed the merge and want to undo
git reset --hard HEAD~1    # Go back before the merge commit

# Nuclear reset: throw away all local changes and match remote exactly
git fetch origin
git reset --hard origin/main
git clean -fd
```

---

12. Best Practices & Common Mistakes

Industry Best Practices

1. **Commit early, commit often** — small commits are easier to review and revert.
2. **Never commit directly to** `main` — always use PRs, even for small fixes.
3. **Keep your branch short-lived** — aim to merge within 1-2 days. Longer = more conflicts.
4. **Rebase before raising a PR** — always be on top of main.
5. **Write the PR description like documentation** — future you will thank you.
6. **Use** `--force-with-lease` **instead of** `--force` — prevents overwriting others' pushes.
7. **Tag releases** — `git tag -a v2.4.0 -m "Release v2.4.0"`.
8. **Use** `.gitignore` **properly** — never commit `.env`, build artifacts, or IDE settings.
9. **Review your own diff before pushing** — `git diff HEAD` or `git log -p`.
10. **Atomic commits** — one logical change per commit.

Common Mistakes and How to Avoid Them

Mistake	Why It's Bad	Fix
Committing `.env` or secrets	Security breach	Use `.gitignore`, rotate credentials immediately
Force-pushing shared branches	Destroys teammates' work	Only force-push your own branches, use `--force-with-lease`
Giant PRs (1000+ lines)	Impossible to review properly	Split into smaller PRs
Meaningless commit messages	Useless git history	Follow Conventional Commits
Not pulling before pushing	Diverged branches	`git pull --rebase` before push
Rebasing shared branches	Rewrites public history	Only rebase local/personal branches
Using `git reset --hard` on pushed commits	Breaks teammates' local repos	Use `git revert` instead
Ignoring test failures in CI	Broken code merges to main	Block merges unless CI passes
Long-lived feature branches	Massive merge conflicts	Merge/PR frequently, use feature flags
Not using `git stash` before switching	Lose or accidentally commit WIP	Always stash before context switching

---

13. Command Cheat Sheet

Daily Commands

Command	What it does
`git status`	Show working directory and staging state
`git diff`	Show unstaged changes
`git diff --staged`	Show staged changes
`git log --oneline --graph`	Compact visual history
`git log --oneline -10`	Last 10 commits
`git fetch origin`	Download remote changes without merging
`git pull --rebase`	Pull and rebase instead of merge
`git push -u origin <branch>`	Push and set upstream

Branch Management

Command	What it does
`git branch`	List local branches
`git branch -a`	List all branches (local + remote)
`git checkout -b <name>`	Create and switch to new branch

Command	What it does
`git switch <name>`	Switch branch (modern alternative)
`git branch -d <name>`	Delete local branch (safe)
`git branch -D <name>`	Force delete local branch
`git push origin --delete <name>`	Delete remote branch
`git branch -m <old> <new>`	Rename branch

Staging & Committing

Command	What it does
`git add -p`	Interactively stage hunks
`git add .`	Stage all changes
`git commit --amend`	Edit last commit (message + content)
`git commit --amend --no-edit`	Amend last commit without changing message
`git reset HEAD <file>`	Unstage a file
`git restore <file>`	Discard changes in working dir
`git restore --staged <file>`	Unstage file (modern syntax)

History & Inspection

Command	What it does
`git log --author="Name"`	Filter commits by author
`git log --since="2 weeks ago"`	Filter by time
`git log -- <file>`	History of specific file
`git show <hash>`	Show commit details and diff
`git diff <branch1>..<branch2>`	Diff between two branches
`git diff <branch1>...<branch2>`	Diff from common ancestor
`git blame <file>`	Show who changed each line
`git shortlog -sn`	Commit count per author

Merge, Rebase & Cherry-pick

Command	What it does
`git merge <branch>`	Merge branch into current
`git merge --no-ff <branch>`	Merge with merge commit always
`git merge --squash <branch>`	Squash all commits into staging
`git rebase <branch>`	Rebase current onto branch
`git rebase -i HEAD~N`	Interactive rebase last N commits
`git rebase --continue`	Continue after resolving conflict

Command	What it does
`git rebase --abort`	Abort rebase entirely
`git cherry-pick <hash>`	Apply a specific commit
`git cherry-pick --no-commit <hash>`	Apply changes without committing

Undo & Recovery

Command	What it does
`git reset --soft HEAD~1`	Undo last commit, keep staged
`git reset --mixed HEAD~1`	Undo last commit, keep unstaged
`git reset --hard HEAD~1`	Undo last commit, discard changes
`git revert <hash>`	Create new commit that undoes target
`git reflog`	See full HEAD movement history
`git stash`	Stash current changes
`git stash pop`	Apply latest stash and drop it
`git stash list`	List all stashes
`git clean -fd`	Remove untracked files + directories

Remote Operations

Command	What it does
`git remote -v`	Show remote URLs
`git remote add origin <url>`	Add remote
`git push --force-with-lease`	Force push (safe version)
`git fetch --prune`	Fetch and remove stale remote refs
`git clone --depth 1 <url>`	Shallow clone (faster for CI)
`git push origin <tag>`	Push a tag to remote
`git push origin --tags`	Push all tags

Tagging

Command	What it does
`git tag`	List all tags
`git tag v2.4.0`	Create lightweight tag
`git tag -a v2.4.0 -m "message"`	Create annotated tag
`git push origin v2.4.0`	Push specific tag
`git tag -d v2.4.0`	Delete local tag
`git push origin --delete v2.4.0`	Delete remote tag

Config

Command	What it does
`git config --global user.name "Name"`	Set username
`git config --global user.email "email"`	Set email
`git config --global core.editor "code --wait"`	Set VS Code as editor
`git config --list`	View all config
`git config --global alias.lg "log --oneline --graph --all"`	Create alias

Quick Reference: Safe vs Dangerous Commands

Command	Safe?	Notes
`git revert`	✅ Always safe	Creates new commit, no history rewrite
`git reset --soft/--mixed`	✅ Safe locally	Don't use on pushed commits
`git reset --hard`	⚠️ Destructive locally	Loses uncommitted work
`git push --force-with-lease`	⚠️ Use carefully	OK on personal branches
`git push --force`	❌ Dangerous	Can overwrite others' work
`git rebase` (local branch)	✅ Safe	Fine on personal, local branches
`git rebase` (shared branch)	❌ Dangerous	Rewrites public history
`git clean -fd`	⚠️ Destructive	Permanently deletes untracked files
`git stash`	✅ Safe	Easy to recover
`git cherry-pick`	✅ Safe	Creates new commit

Last updated: 2025 | Maintained by: Backend Platform Team